Bitácora S09

Mexi Banco: mensajería con RabbitMQ, idempotencia, DLQ y Outbox

Estudiante	Victoria Muñoz
Rama	entregas_munoz_victoria
Fecha de ejecución	24 de octubre de 2026
Pruebas	transferencia: 18/18; notificacion: 1/1

Objetivo

Desacoplar la transferencia de la notificación mediante eventos durables, evitar avisos duplicados con eventoId, apartar mensajes ilegibles en una DLQ y conservar eventos pendientes con un Transactional Outbox.

Nota de evidencia

Las capturas de comandos y resultados incluidos aquí corresponden a una ejecución real de Docker y RabbitMQ. La Parte 0 del comportamiento anterior y el duplicado antes del arreglo no se volvieron a ejecutar en esta sesión sobre el código base; por eso se describen como comportamiento previo y no se presentan como una captura inventada.

1. El problema

En la versión base, la saga ya había cargado la cuenta origen, abonado la cuenta destino y registrado el ledger, pero todavía llamaba a notificacion por HTTP antes de marcar la transferencia como COMPLETADA. Si notificacion estaba caída, la llamada lanzaba una excepción después de mover el dinero. La fila quedaba en ABONADA y el cliente recibía error aunque el dinero ya había llegado.

Frase de la causa

> La saga esperaba una respuesta síncrona de notificacion después de mover el dinero, así que una falla de disponibilidad convirtió un aviso no crítico en un error de la transferencia.

Evidencia de la versión nueva

```
TRANSFERENCIA_11 201 ... estado: COMPLETADA
TRANSFERENCIA_12 201 ... estado: COMPLETADA
TRANSFERENCIA_13 201 ... estado: COMPLETADA
```

2. Publicar el evento

Cada transferencia recibe un eventoId al crear la fila. El evento es un record separado de la entidad y contiene el contrato público: tipo, transferenciaId, CLABEs, monto, saldos y fecha. El publicador usa el exchange topic transferencias, la routing key correspondiente, mensaje persistente y MessageId igual al eventoId.

Publisher confirm

publisher-confirm-type: correlated hace que RabbitMQ confirme o rechace cada publicación. mandatory=true permite detectar que ninguna cola recibió el mensaje; la entrega persistente permite que el mensaje sobreviva junto con una cola durable. Esto garantiza que el broker aceptó el mensaje o que la aplicación detectó el fallo. No garantiza por sí solo que la base y RabbitMQ formen una sola transacción; esa es la razón del Outbox.

Evidencia de arranque

```
transferencia ... Up ... healthy
notificacion  ... Up ... healthy
rabbitmq      ... Up ... healthy

La respuesta incluyó eventoId: a3b2b1d0-1de9-47d8-86b4-b706f2dc776a
```

3. Desacople en el tiempo

Con notificacion detenida, se enviaron tres transferencias. La saga devolvió 201 y COMPLETADA porque dejó eventos en el exchange y la cola durable conservó los mensajes. Al volver notificacion, los mensajes se consumieron y se generaron los avisos de Ana y Beto.

Productor y cola

```
TRANSFERENCIA_11 201 ... COMPLETADA
TRANSFERENCIA_12 201 ... COMPLETADA
TRANSFERENCIA_13 201 ... COMPLETADA

notificacion.transferencias     3 Ready  0 Unacked
```

Después de encender notificacion

```
notificacion.transferencias     0 Ready  0 Unacked

Beto: TRANSFERENCIA_RECIBIDA por 11.00, 12.00 y 13.00
Ana: TRANSFERENCIA_ENVIADA por 11.00, 12.00 y 13.00
```

TransferenciaRevertida

El consumidor también reconoce transferencia.revertida y guarda DEVOLUCION_TRANSFERENCIA para la cuenta origen. La prueba real de consumo produjo:

Aviso de devolución

```
REVERTIDA_EVENT 200 {"routed":true}
DEVOLUCION_TRANSFERENCIA por 150, saldo resultante null
```

4. El duplicado

El ack manual se realiza después de guardar. Si el proceso muere entre ambas operaciones, RabbitMQ vuelve a entregar el mensaje. El arreglo registra eventoId en evento_procesado con una restricción única y un INSERT ... ON CONFLICT DO NOTHING dentro de la misma transacción que saveAll.

Resultado después del arreglo

Se republicó el mismo cuerpo con el mismo eventoId a3b2b1d0-1de9-47d8-86b4-b706f2dc776a. El consumidor aceptó el mensaje, pero la marca única detectó que ya existía y devolvió cero avisos nuevos.

Evento repetido

```
DUPLICATE_PUBLISH 200 {"routed":true}

[consumidor] TransferenciaRealizada de la transferencia 8
(evento a3b2b1d0-1de9-47d8-86b4-b706f2dc776a): 0 aviso(s) nuevo(s)
```

Nota sobre el antes

El comportamiento anterior está documentado en el enunciado y en el código base: sin la tabla de procesados, dos entregas del mismo evento producían dos juegos de avisos. No presento una captura de ese estado porque no se ejecutó nuevamente antes del arreglo en esta sesión.

5. Mensaje envenenado y DLQ

La cola principal se declaró con dead-letter-exchange transferencias.dlx y dead-letter-routing-key notificacion.transferencias. El consumidor captura el JSON ilegible y responde basicNack con requeue=false. RabbitMQ mueve el mensaje a notificacion.transferencias.dlq.

DLQ

```
POISON_PUBLISH 200 {"routed":true}

notificacion.transferencias.dlq   1 Ready  0 Unacked
notificacion.transferencias       0 Ready  0 Unacked
```

Una transferencia normal posterior respondió 201 y COMPLETADA; el mensaje ilegible no bloqueó la cola principal.

Normal después del veneno

```
201 {"id":11,"monto":14,"estado":"COMPLETADA","eventoId":"eae103b6-0657-4888-bde7-1bc80b72338f"}
```

6. Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso?

En esta versión el estado COMPLETADA y el evento pendiente se guardan juntos en la tabla outbox. Mientras RabbitMQ está caído, el relay no puede publicar y deja la fila pendiente; el aviso todavía no llega, pero no se pierde. El publisher confirm no arregla por sí solo el problema porque solo confirma una publicación cuando RabbitMQ está disponible; no puede hacer una transacción atómica entre la base y el broker.

2. ¿Por qué el ack va después de guardar?

Si el ack va después de guardar, una muerte del consumidor antes del ack provoca una redelivery, pero el mensaje no se pierde; la idempotencia evita que esa repetición vuelva a crear avisos. Si el ack fuera antes de guardar y el proceso muriera después del ack, RabbitMQ borraría el mensaje y el aviso se perdería. Por eso se elige guardar primero y confirmar después.

3. Elección y sacrificio

Elección: quitar la llamada HTTP síncrona y publicar un evento. Porque: el aviso es secundario y no debe tumbar una transferencia cuyo dinero ya se movió. Sacrifico: la respuesta ya no confirma que el cliente recibió el aviso en ese instante; gano disponibilidad y acepto consistencia eventual.

7. Reto Outbox

El Outbox evita la doble escritura entre la base y RabbitMQ. OutboxService guarda la transferencia final y el evento en la misma transacción. OutboxRelay consulta pendientes con SELECT ... FOR UPDATE SKIP LOCKED, publica con publisher confirm y marca publicado solo después del ack del broker. Si el relay muere después de publicar pero antes de marcar, el mismo eventoId puede salir otra vez y el consumidor lo absorbe idempotentemente.

RabbitMQ detenido

```
RABBIT_OFF_TRANSFER 201
{"id":14,"monto":1,"estado":"COMPLETADA","eventoId":"a848beeb-3106-43e2-9e59-5a1a3ec30d7a"}

Outbox: no se pudo publicar ... Connection refused; queda pendiente
```

RabbitMQ recuperado

```
Outbox: evento a848beeb-3106-43e2-9e59-5a1a3ec30d7a publicado

Beto: TRANSFERENCIA_RECIBIDA por 1.00
Cola principal: 0 Ready, 0 Unacked
```

Conclusión

El productor ya no depende de que notificacion esté vivo en el mismo instante. La cola durable resuelve el desacople temporal, la idempotencia resuelve las redeliveries, la DLQ aparta mensajes imposibles de procesar y el Outbox resuelve el caso en que el broker cae justo cuando la saga termina.